POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY

Intelligent System Development

ASSIGNMENT 05

Comparative Empirical Evaluation of Convolutional Depth and Framework Implementations

Systematic Investigation of CNN-3 versus CNN-5 in TensorFlow and PyTorch across SVHN, GTSRB, and Tabular Clinical Benchmarks

---

Instructor / Lecturer: Assoc. Prof. Dinh Que Tran, Ph.D.
Student Author: **Bùi Nguyên Hoàng Việt**
Student ID: **B23DCDT285**
Class: E23CNPM02
Academic Year: 2025 – 2026
Project Repository: https://github.com/minorin2847/INTEL_SYS_DEV_2026

Hanoi, 2026

Contents

1 Executive Summary and Benchmark Overview **2**

2 Theoretical Foundations and Convolutional Mechanics **3**
2.1 2D Convolution as Spatial Feature Filtering . . . 3
2.2 Activation, Normalization, Pooling, and Regularization . . . 4
2.3 Loss Formulations and Optimization Dynamics . . . 5
2.4 Analytical Backpropagation: A Hand-Calculated Gradient Step . . . 6
2.5 Procedural Lifecycle of a Training Epoch . . . 7
2.6 Operational Duality: Training Mode vs. Evaluation Mode . . . 7
2.7 1D Convolutions on Tabular Datasets . . . 8
2.8 Evaluation Metrics Under Class Imbalance . . . 8

3 Experimental Methodology and Experimental Controls **9**

4 Dataset Profiling and Preprocessing Pipelines **9**
4.1 SVHN (Street View House Numbers) . . . 9
4.2 GTSRB (German Traffic Sign Recognition Benchmark) . . . 11
4.3 Diabetes (Playground Series S5E12) . . . 13

5 Network Architectures and Implementation Models **15**
5.1 Implementation Paradigm: Keras vs. PyTorch . . . 16

6 Empirical Results and Diagnostic Evaluations **16**
6.1 SVHN Benchmark Results . . . 16
6.2 GTSRB Benchmark Results . . . 18
6.3 Diabetes Benchmark Results . . . 20

7 Cross-Dataset Comparative Synthesis **22**
7.1 Framework Invariance Verification . . . 23
7.2 Why Depth Powers Computer Vision: Receptive Field Growth . . . 23
7.3 Why Depth Fails on Tabular Records: The Inductive Mismatch . . . 23

8 Conclusions and Experimental Limitations **24**

A Appendix: Reproducibility and Additional Diagnostics **24**
A.1 Additional Diagnostic Visualizations . . . 24
A.2 Project Reproduction and Execution Guide . . . 25

1 Executive Summary and Benchmark Overview

This report presents a systematic empirical evaluation of convolutional neural network depth and deep learning framework implementations. We construct, train, and evaluate 3-layer and 5-layer convolutional architectures (`CNN-3` and `CNN-5`) across both **TensorFlow/Keras** (`TF`) and **PyTorch** (`PT`) on three distinct machine learning benchmarks:

- **Street View House Numbers (SVHN):** 99,289 real-world cropped digit photos ($32 \times 32 \times 3$), 10 classes (0–9) featuring distracting flanking characters and natural illumination artifacts.
- **German Traffic Sign Recognition Benchmark (GTSRB):** 51,839 real-world road sign images ($32 \times 32 \times 3$) distributed across 43 classes, presenting severe class imbalance ($10.71\times$ max/min ratio) and low-contrast shadow conditions.
- **Diabetes Health Indicators (Kaggle Playground Series S5E12):** 300,000 patient records comprising 24 clinical attributes mapped to 42 numerical inputs, serving as an unordered tabular classification task.

Every experimental run enforces identical data partitions (a fixed 70/15/15 stratified split), matched parameter counts, identical weight initializations (Glorot uniform), aligned Batch Normalization momentum (0.9 in Keras $\equiv$ 0.1 in PyTorch), identical Adam hyperparameters ($\eta = 10^{-3}, \epsilon = 10^{-7}$), a mini-batch size of 256, and identical early-stopping patience rules with seed 42. In addition, an unregularized Multi-Layer Perceptron (`DB-TF-MLP`) is trained on the diabetes dataset as an architectural reference baseline.

Table 1: Master summary across all 13 evaluated model runs on the test set. s/epoch denotes the median training duration per epoch on CPU; ms/1k denotes inference latency per 1,000 predictions.

Run ID	Framework	Model	Params	Accuracy	Macro F1	Epochs	s/epoch	ms / 1k
`SV-TF-CNN-3`	TensorFlow	CNN-3	129,290	0.9117	0.9072	20	45.0s	0.10
`SV-TF-CNN-5`	TensorFlow	CNN-5	175,658	0.9306	0.9265	10	86.5s	0.22
`SV-PT-CNN-3`	PyTorch	CNN-3	129,290	0.9106	0.9046	18	48.9s	0.27
`SV-PT-CNN-5`	PyTorch	CNN-5	175,658	0.9294	0.9250	13	98.7s	0.49
`GT-TF-CNN-3`	TensorFlow	CNN-3	137,771	0.9781	0.9751	20	23.3s	0.12
`GT-TF-CNN-5`	TensorFlow	CNN-5	184,139	0.9911	0.9907	11	45.5s	0.22
`GT-PT-CNN-3`	PyTorch	CNN-3	137,771	0.9848	0.9843	20	25.9s	0.28
`GT-PT-CNN-5`	PyTorch	CNN-5	184,139	**0.9943**	**0.9937**	16	52.8s	0.50
`DB-TF-CNN-3`	TensorFlow	CNN-3	191,169	**0.6465**	**0.6310**	7	12.8s	0.08
`DB-TF-CNN-5`	TensorFlow	CNN-5	391,329	0.6333	0.6259	5	28.8s	0.16
`DB-PT-CNN-3`	PyTorch	CNN-3	191,169	0.6371	0.6280	7	20.1s	0.12
`DB-PT-CNN-5`	PyTorch	CNN-5	391,329	0.6446	0.6308	8	42.5s	0.24
`DB-TF-MLP`	TensorFlow	MLP	**6,977**	0.6275	0.6222	5	**1.1s**	0.03

Key Strategic Findings

1. **Spatial Depth Powers Visual Recognition:** On structured 2D images, deepening the network from 3 to 5 convolutional layers yields substantial accuracy and macro F1 gains across both frameworks (+1.89% accuracy on SVHN, +1.30% on GTSRB). The deeper hierarchy expands the effective receptive field from 22×22 to 28×28 pixels, enabling the network to resolve subtle morphological confusions between look-alike digits (e.g., 7 vs. 1, 5 vs. 3) and fine-grained traffic sign pictograms. The overall top-performing model is `GT-PT-CNN-5`, attaining 99.43% test accuracy and 0.9937 macro F1 across 43 classes.
2. **The Tabular Convolution Paradox:** On the tabular diabetes dataset, extra convolutional depth provides zero performance gain. CNN-5 fails to outperform CNN-3 across all metrics (ROC-AUC remains static at 0.694–0.696), despite doubling parameter counts to 391,329 and increasing compute time by 2.2×. Most strikingly, a compact 2-layer MLP with only 6,977 parameters matches the predictive ROC-AUC of the 391k-parameter CNN-5 (0.6948 vs. 0.6957) while training in just 1.1 seconds per epoch (26× faster). Because tabular features possess no spatial continuity, local 1D convolutions enforce artificial proximity constraints that provide no legitimate inductive advantage.
3. **Strict Empirical Parity Across Frameworks:** When experimental parameters, initialization seeds, and mini-batch structures are matched, TensorFlow and PyTorch produce mathematically equivalent models. Test accuracy discrepancies between corresponding TF and PT models remain below 0.011 across all benchmarks, confirming that deep learning libraries function as computational backends that do not alter the underlying loss landscape.
4. **Representation Learning vs. Epoch Cost:** While CNN-5 requires roughly 2× more training time per epoch than CNN-3, it extracts richer feature representations earlier in the training trajectory. Consequently, deeper models frequently trigger early stopping in fewer epochs (e.g., 10 epochs for `SV-TF-CNN-5` vs. 20 epochs for `SV-TF-CNN-3`), resulting in comparable or lower total wall-clock training times on image tasks.

2 Theoretical Foundations and Convolutional Mechanics

This section details the mathematical mechanics, layer transformations, and optimization rules that govern convolutional network training, accompanied by clear conceptual analogies and worked numerical examples.

2.1 2D Convolution as Spatial Feature Filtering

A convolutional layer operates as a set of learnable spatial filters sliding systematically across an input tensor. Unlike a fully connected layer where every input unit connects to every output unit, a convolutional filter restricts its connections to a small, localized spatial window (local receptive field) and reuses the exact same weights across all spatial positions (weight sharing / translation equivariance).

Mathematically, given an input tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{in}}$ and a filter bank $\mathbf{W} \in \mathbb{R}^{F \times K \times K \times C_{in}}$ with bias vector $\mathbf{b} \in \mathbb{R}^{F}$, the pre-activation response at spatial location (i, j) for filter f is:

$$\mathbf{Y}(i, j, f) = \sum_{c=1}^{C_{in}} \sum_{u=-\lfloor K/2 \rfloor}^{\lfloor K/2 \rfloor} \sum_{v=-\lfloor K/2 \rfloor}^{\lfloor K/2 \rfloor} \mathbf{X}(i+u, j+v, c) \cdot \mathbf{W}(f, u, v, c) + b(f) \tag{1}$$

The output spatial resolution (H_{out}, W_{out}) is governed by kernel size K, stride S, and padding P:

$$H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1, \qquad W_{out} = \left\lfloor \frac{W + 2P - K}{S} \right\rfloor + 1 \tag{2}$$

All convolutional layers in our 2D image models employ 3×3 kernels ($K = 3$), unit stride ($S = 1$), and **"same" padding** ($P = 1$). Under this configuration, the spatial dimensions of feature maps are perfectly preserved prior to pooling ($32 \times 32 \to 32 \times 32$).

Worked Example: Edge Detection on Digit Strokes. Consider a single-channel 5×5 pixel patch representing a vertical white stroke of a digit centered on a dark background:

$$\mathbf{X} = \begin{bmatrix} 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 \end{bmatrix}, \qquad \mathbf{W} = \begin{bmatrix} -1 & 0 & +1 \\ -1 & 0 & +1 \\ -1 & 0 & +1 \end{bmatrix}, \qquad b = 0$$

When this 3×3 vertical edge filter slides across the patch:

- Centered at column 2 (window spanning columns 1 to 3 with values $[0, 0, 1]$ across all rows):

  $$\mathbf{Y} = 3 \times [(-1)(0) + (0)(0) + (+1)(1)] = +\mathbf{3.0}$$

  The large positive response indicates the detection of a **rising edge** (transition from dark background to the bright digit stroke).
- Centered at column 4 (window spanning columns 3 to 5 with values $[1, 1, 0]$ across all rows):

  $$\mathbf{Y} = 3 \times [(-1)(1) + (0)(1) + (+1)(0)] = -\mathbf{3.0}$$

  The strong negative response signals a **falling edge** (transition from digit stroke back to background).
- Centered over uniform background (columns $[0, 0, 0]$): $\mathbf{Y} = \mathbf{0.0}$ (no feature detected).

2.2 Activation, Normalization, Pooling, and Regularization

Stacking linear convolutions without non-linearities collapses mathematically into a single matrix multiplication. To learn complex hierarchies, convolutions are paired with activation functions, normalization blocks, and pooling layers.

Table 2: Mathematical formulations and worked numerical examples for core network layers.

Layer Type	Mathematical Formulation	Worked Numerical Example
ReLU	$f(x) = \max(0, x)$	Inputs: $[-3.2, 0.0, 4.5] \to [0.0, 0.0, 4.5]$. Eliminates negative activations, providing non-linearity.
Max Pooling (2×2)	$\text{out}(i,j) = \max_{u,v\in\{0,1\}} \mathbf{X}(2i+u, 2j+v)$	Block $\begin{bmatrix} 0.2 & 0.8 \\ 0.4 & 1.5 \end{bmatrix} \to 1.5$. Halves spatial resolution, conferring small translation invariance.
Batch Normalization	$\hat{x} = \frac{x-\mu_B}{\sqrt{\sigma_B^2+\epsilon}}, \quad y = \gamma\hat{x} + \beta$	Batch $[2.0, 4.0, 6.0]$ ($\mu = 4, \sigma = 1.63$) $\to [-1.22, 0.0, +1.22]$. Normalizes activations to stabilize training.
Global Avg Pooling	$\text{GAP}(c) = \frac{1}{HW}\sum_{i=1}^{H}\sum_{j=1}^{W}\mathbf{X}(i,j,c)$	Tensor $(4, 4, 128) \to (128)$. Collapses spatial grids into 1D channel summaries without parameter inflation.
Dropout ($p = 0.3$)	$y = \frac{1}{1-p}\cdot m \odot x, \quad m \sim \text{Bernoulli}(1-p)$	Training: $[1.0, 2.0, 0.5] \to [1.43, 0.0, 0.71]$. Test: $[1.0, 2.0, 0.5]$ (identity). Prevents co-adaptation.

Analytical Parameter Derivations.

- **2D Convolutional Layer:** $(K \cdot K \cdot C_{in} + 1) \cdot F$. For Layer 1 ($3 \times 3$ filter, 3 RGB channels, 32 filters): $(3 \cdot 3 \cdot 3 + 1) \cdot 32 = 28 \cdot 32 = \mathbf{896}$ parameters.
- **Batch Normalization Layer:** $2F$ trainable parameters (γ scale, β shift) plus $2F$ non-trainable tracking running mean and variance.
- **Dense Linear Layer:** $(N_{in} + 1) \cdot N_{out}$. A projection from 128 to 256 units requires $(128 + 1) \cdot 256 = \mathbf{33,024}$ parameters.

2.3 Loss Formulations and Optimization Dynamics

- **Softmax and Multi-Class Cross-Entropy Loss:** Unnormalized network logits $\mathbf{z} \in \mathbb{R}^C$ are converted into probabilities via Softmax:

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}}, \qquad \mathcal{L}_{CE} = -\sum_{k=1}^{C} y_k \ln p_k = -\ln p_{\text{true}} \tag{3}$$

  Worked Example: Suppose for a digit image the true class is "3" ($y_3 = 1$). If the model outputs logits $z_3 = 2.0$, $z_8 = 1.0$, and all other 8 logits are 0.0:

$$\sum e^{z_j} = e^{2.0} + e^{1.0} + 8 \cdot e^{0.0} = 7.389 + 2.718 + 8.0 = 18.107$$

$$p(3) = \frac{7.389}{18.107} \approx 0.408 \implies \mathcal{L}_{CE} = -\ln(0.408) \approx \mathbf{0.896}$$

  Prior to training, a randomly initialized model outputs near-zero logits, assigning uniform probability $1/C$ across all classes. The theoretical starting loss is $\ln(10) \approx \mathbf{2.303}$ on SVHN and $\ln(43) \approx \mathbf{3.761}$ on GTSRB.

- **Binary Cross-Entropy Loss (Diabetes):** For clinical binary diagnosis:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad \mathcal{L}_{BCE} = -[y \ln p + (1 - y) \ln(1 - p)] \tag{4}$$

- **Adam Optimization Algorithm:** Maintains running estimates of gradient momentum (m_t) and uncentered variance (v_t):

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \quad \theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{5}$$

  with learning rate $\eta = 10^{-3}$, decay factors $\beta_1 = 0.9, \beta_2 = 0.999$, and numerical stability constant $\epsilon = 10^{-7}$.

- **Early Stopping Protocol:** Validation loss is evaluated after every epoch. If validation loss does not achieve a new minimum for 3 consecutive epochs (`patience = 3`), training halts, and the model weights from the optimal epoch are restored.

2.4 Analytical Backpropagation: A Hand-Calculated Gradient Step

To demystify how gradients update weights, we trace a complete hand calculation for a single neuron with input $x = 2.0$, weight $w = 0.5$, bias $b = 0.0$, and true target $y = 1$.

1. Forward Pass:

$$z = w \cdot x + b = (0.5)(2.0) + 0.0 = 1.0$$

$$p = \sigma(z) = \frac{1}{1 + e^{-1.0}} \approx 0.7311 \implies \mathcal{L} = -\ln(0.7311) \approx \mathbf{0.3132}$$

2. Backward Pass via Chain Rule: For sigmoid activation combined with binary cross-entropy loss, the derivative simplifies cleanly:

$$\frac{\partial \mathcal{L}}{\partial z} = p - y = 0.7311 - 1.0 = \mathbf{-0.2689}$$

Applying the chain rule to compute parameter gradients:

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial w} = (-0.2689) \cdot x = (-0.2689)(2.0) = \mathbf{-0.5378}$$

$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z} \cdot \frac{\partial z}{\partial b} = (-0.2689) \cdot (1.0) = \mathbf{-0.2689}$$

3. Gradient Descent Weight Update (with learning rate $\eta = 0.1$):

$$w_{\text{new}} = w - \eta \frac{\partial \mathcal{L}}{\partial w} = 0.5 - (0.1)(-0.5378) = \mathbf{0.5538}$$

$$b_{\text{new}} = b - \eta \frac{\partial \mathcal{L}}{\partial b} = 0.0 - (0.1)(-0.2689) = \mathbf{0.0269}$$

4. Mathematical Verification: Re-evaluating the neuron with the updated parameters:

$$z_{\text{new}} = (0.5538)(2.0) + 0.0269 = 1.1345 \implies p_{\text{new}} = \sigma(1.1345) \approx \mathbf{0.7567}$$

$$\mathcal{L}_{\text{new}} = -\ln(0.7567) \approx \mathbf{0.2788}$$

The loss decreased monotonically from 0.3132 to 0.2788, and the predicted probability moved closer to the true target 1.0.

Table 3: Analytical backward gradient flow rules across different layer types.

Layer Type	Analytical Backward Gradient Flow Equation
Softmax + CE	$\frac{\partial \mathcal{L}}{\partial z_k} = p_k - y_k$. Difference between predicted probability and one-hot ground truth.
Dense Layer	$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \mathbf{x}^T \cdot \frac{\partial \mathcal{L}}{\partial \mathbf{z}}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{x}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}} \cdot \mathbf{W}^T$. Matrix multiplication routes errors back to preceding layers.
ReLU Activation	$\frac{\partial \mathcal{L}}{\partial x} = \frac{\partial \mathcal{L}}{\partial y} \cdot \mathbb{I}(x > 0)$. Passes gradient unchanged if positive; blocks gradient completely if inactive.
Max Pooling	$\frac{\partial \mathcal{L}}{\partial x(i,j)} = \frac{\partial \mathcal{L}}{\partial y} \cdot \mathbb{I}(x(i,j) = \max)$. Routes the entire incoming gradient to the single winning pixel.
2D Convolution	$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum \mathbf{X}_{\text{window}} * \frac{\partial \mathcal{L}}{\partial \mathbf{Y}}$. Accumulates gradient contributions across all sliding positions.

2.5 Procedural Lifecycle of a Training Epoch

An epoch represents one complete pass over the training set. In PyTorch, this is executed as an explicit procedural loop:

Table 4: The 8 procedural phases executed during each training epoch.

Step	PyTorch Code	Mechanistic Function
1	`model.train()`	Sets training mode: enables Dropout and activates mini-batch statistics in Batch Normalization.
2	`for x, y in loader:`	Iterates through random batches of size 256 produced by a shuffled DataLoader.
3	`optimizer.zero_grad()`	Resets residual gradient buffers from the previous mini-batch.
4	`logits = model(x)`	Forward pass: passes the tensor through convolutional and linear layers.
5	`loss = criterion(logits, y)`	Calculates scalar mini-batch loss via cross-entropy.
6	`loss.backward()`	Backpropagation: populates parameter '.grad' attributes via the chain rule.
7	`optimizer.step()`	Updates model weights using Adam momentum and adaptive second moments.
8	`model.eval()` + Val Loop	Evaluates loss on validation set; checks early stopping criteria.

Mini-Batch Counts per Epoch: SVHN (69,502 training samples) requires 272 updates per epoch; GTSRB (36,287 training samples) requires 142 updates; Diabetes (210,000 training samples) requires 821 updates.

2.6 Operational Duality: Training Mode vs. Evaluation Mode

Batch Normalization and Dropout alter their core behaviors depending on execution mode:

Table 5: Behavioral comparison of layers between training and evaluation phases.

Layer Type	Training Mode (`model.train()`)	Evaluation Mode (`model.eval()`)
BatchNorm	Normalizes using the current mini-batch mean μ_B and variance σ_B^2; updates running population estimates: $\mu_{\text{run}} = 0.9\mu_{\text{run}} + 0.1\mu_B$.	Freezes running statistics; normalizes using stored population parameters $\mu_{\text{run}}, \sigma^2_{\text{run}}$, ensuring single-sample determinism.
Dropout	Randomly sets 30% of activations to zero; scales surviving activations by $1/(1-0.3) \approx 1.428$ to preserve energy.	Disabled entirely; functions as an exact identity pass-through operator.

Failing to call `model.eval()` during testing corrupts predictions by injecting random dropout mask noise and subjecting individual samples to unrepresentative batch-dependent normalization shifts.

2.7 1D Convolutions on Tabular Datasets

For the diabetes dataset, each patient is represented as an ordered vector of $d = 42$ standardized numerical and one-hot categorical features, formatted as a 1D tensor: $(N, 1, 42)$. A 1D convolutional filter of width $K = 3$ slides across adjacent feature indices:

$$\text{out}[i] = \sum_{k=-1}^{1} w[k] \cdot x[i+k] + b \tag{6}$$

While effective for temporal audio signals or natural language text where neighboring elements possess physical temporal or syntactic proximity, **tabular columns possess no natural spatial topology**. Grouping arbitrary adjacent columns (such as `age`, `screen_time`, and `bmi`) forces local receptive fields onto non-local phenomena. Furthermore, Global Average Pooling cannot be applied to tabular data without erasing feature identities; tabular CNNs must therefore employ **Flatten** layers, concentrating over 80% of their total parameters in the dense classification head.

2.8 Evaluation Metrics Under Class Imbalance

- **Precision, Recall, and F1-Score:** Precision $= \frac{TP}{TP+FP}$, Recall $= \frac{TP}{TP+FN}$, F1 $= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$.
- **The Deception of Raw Accuracy:** Under severe class imbalance, accuracy is misleading. Consider a test set with 95 samples of Class A and 5 samples of Class B. A naive classifier that predicts Class A for all inputs achieves 95% accuracy, but a recall of 0% on Class B and a Macro F1 of only 0.487.
- **Macro F1 Score:** Assigns equal weight to each class regardless of sample count:

  $$\text{Macro F1} = \frac{1}{C} \sum_{c=1}^{C} \text{F1}_c$$

  Macro F1 serves as our primary quality metric on GTSRB (10.71× class imbalance) and SVHN.
- **ROC-AUC:** Measures the probability that a randomly chosen diabetic patient receives a higher predicted probability than a randomly chosen non-diabetic patient (0.5 represents random guessing; 1.0 represents perfect separation).

3 Experimental Methodology and Experimental Controls

The project repository is structured into self-contained operational components:

- **Git Repository:** https://github.com/minorin2847/INTEL_SYS_DEV_2026
- `notebook/`: Contains the self-contained execution notebooks (`sv_svhn.ipynb`, `gt_gtsrb.ipynb`, and `db_diabetes.ipynb`), alongside subdirectories `models/` (serialized checkpoints) and `results/` (logged metric JSON artifacts).
- `dataset/`: Houses dataset archives, preprocessing instructions, and pre-computed stratified splits (`splits.npz`).
- `report/`: Contains the LaTeX documentation (`Assignment_05.tex` and `Assignment_05.pdf`).

Experimental Controls for Strict Framework Parity. To eliminate confounding variables, both frameworks were subjected to identical training conditions:

- **Identical Stratified Split:** A fixed 70/15/15 train/validation/test stratified split was saved as `splits.npz` and loaded identically by both TensorFlow and PyTorch.
- **Parameter Alignment:** Glorot uniform (Xavier uniform) weight initialization with zero biases was enforced in PyTorch to match Keras defaults. BatchNorm momentum was set to 0.9 in Keras ($\equiv$ 0.1 in PyTorch). Total trainable parameter counts were programmatically asserted to be bit-identical between frameworks.
- **Optimization Alignment:** Standard Adam ($\eta = 10^{-3}, \epsilon = 10^{-7}$) was utilized across all runs with a batch size of 256.
- **Execution Environment:** Benchmarks were executed on an AMD CPU architecture under native Windows. Timings represent pure CPU compute performance.

4 Dataset Profiling and Preprocessing Pipelines

Table 6: Dataset characteristics, sample counts, and input dimensionality.

Dataset	Source Domain	Total Samples	Classes	Input Dimensionality
`SV` SVHN	Street View House Numbers	99,289	10	$32 \times 32 \times 3$ RGB image
`GT` GTSRB	German Traffic Signs	51,839	43	$32 \times 32 \times 3$ RGB image
`DB` Diabetes	Kaggle Playground S5E12	300,000 (sample)	2	24 features $\rightarrow$ 42 inputs

4.1 SVHN (Street View House Numbers)

Loaded from MATLAB `.mat` files via `scipy.io.loadmat`, SVHN comprises cropped house digits where the target digit is centered, but distracter digits often encroach on borders. Label 10 is remapped to 0. Digit frequencies range from 13,272 (digit 1) to 4,378 (digit 9), representing an imbalance ratio of 3.03. Pixels are normalized to $[0, 1]$ via division by 255.0.

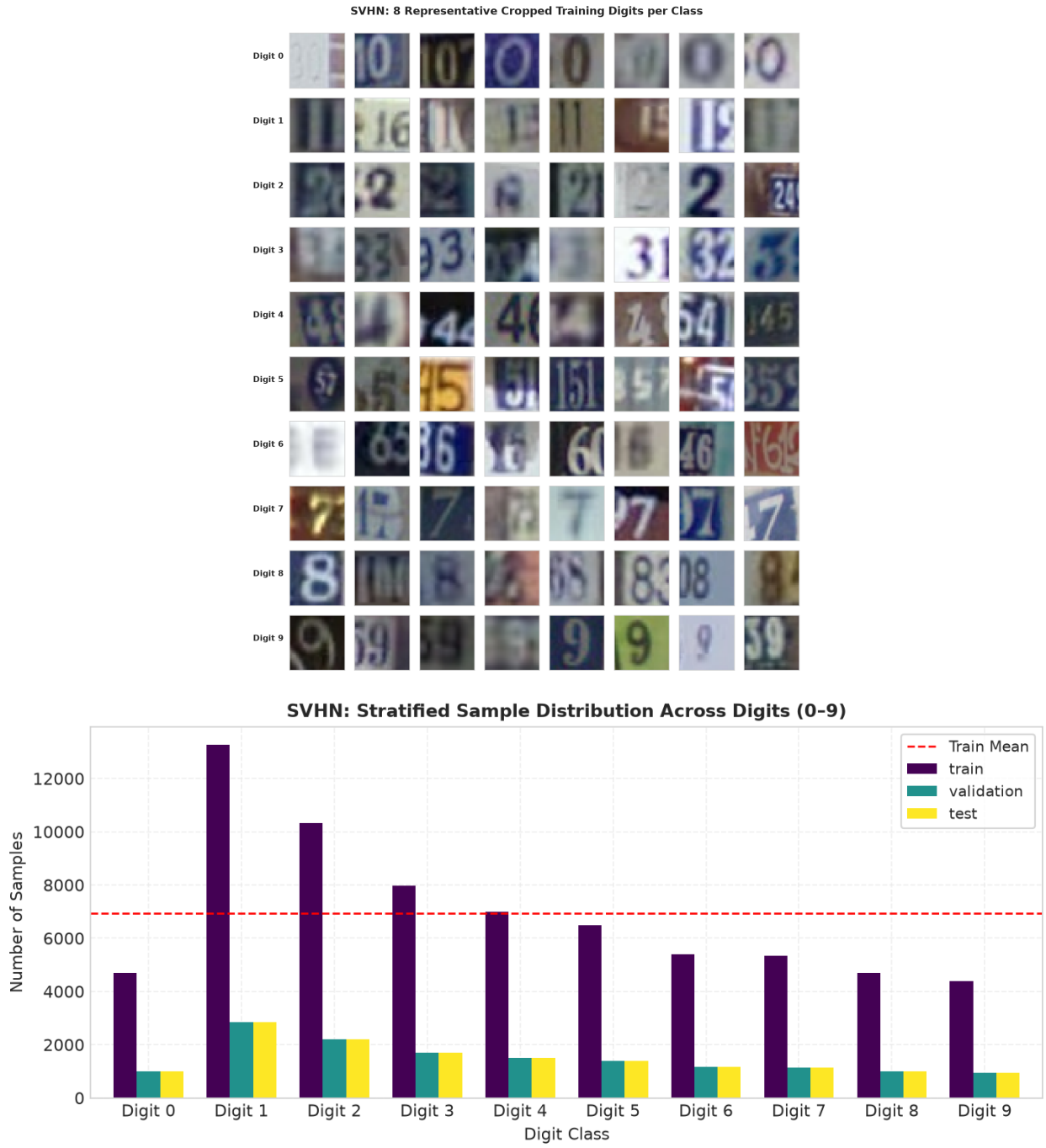


Figure 1: `SV`: representative cropped training samples per digit (top) and stratified sample distribution across splits (bottom).

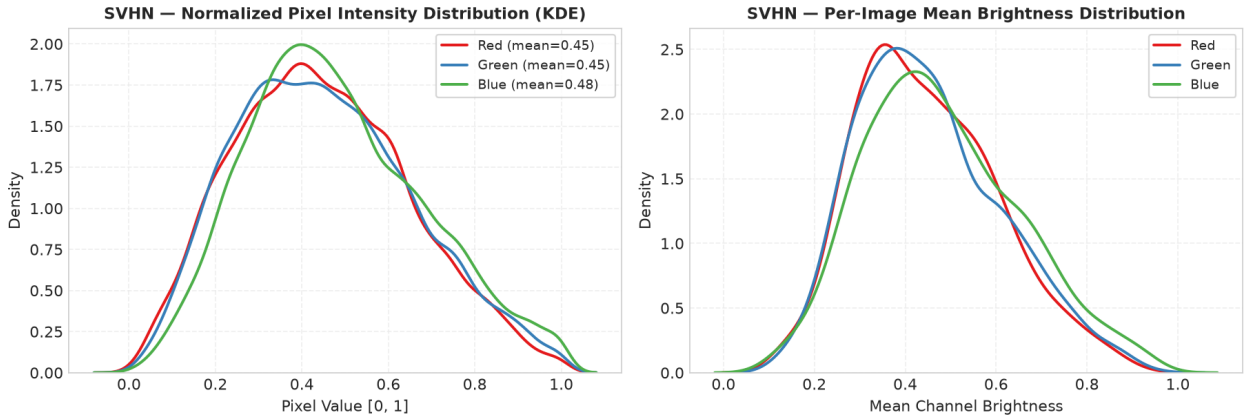


Figure 2: `SV`: 3-channel pixel intensity distribution via KDE (left) and per-image brightness distribution (right).

4.2 GTSRB (German Traffic Sign Recognition Benchmark)

Compiled from 43 classes of German road signs, GTSRB exhibits severe class imbalance, with sample counts ranging from 210 (Speed limit 20) to 2,250 (Speed limit 50), a ratio of 10.71. Many photos suffer from intense shadows, motion blur, and underexposure. Normalizing inputs and applying Batch Normalization is crucial to ensure stable gradient updates across high-contrast conditions.

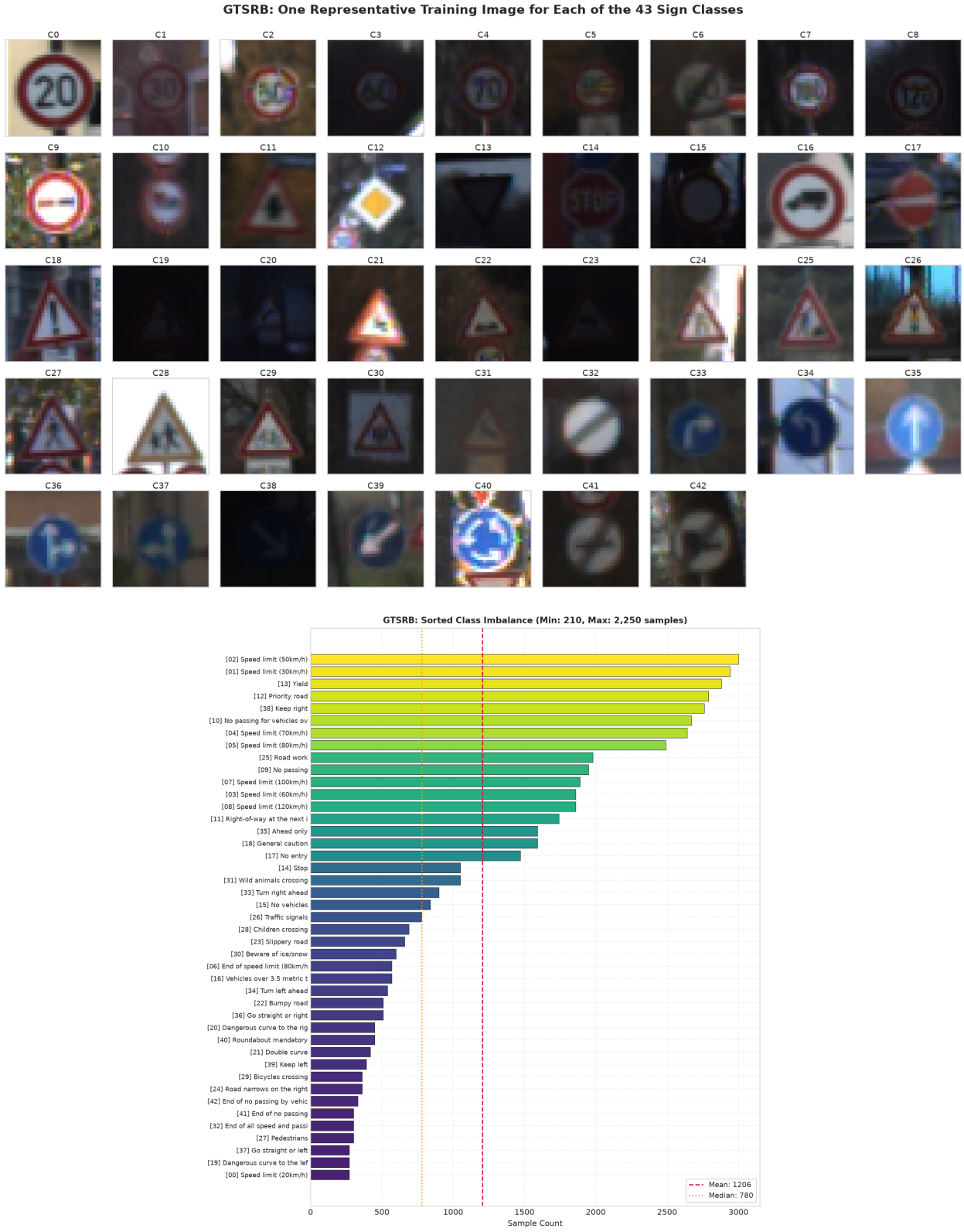


Figure 3: `GT`: one representative training sign for each of the 43 classes (top) and sorted horizontal sample distribution showing class imbalance (bottom).

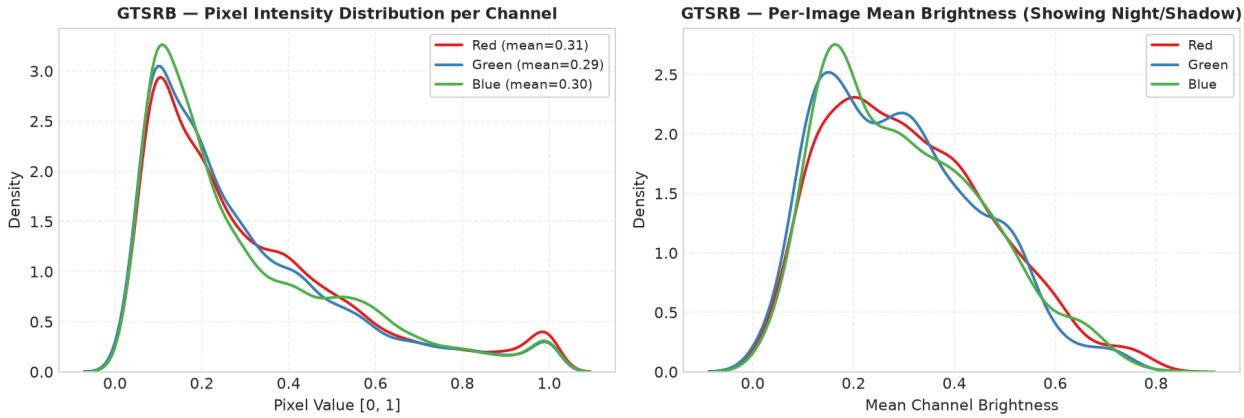


Figure 4: `GT`: 3-channel pixel intensity distribution via KDE (left) and per-image brightness distribution showing night/shadow effects (right).

4.3 Diabetes (Playground Series S5E12)

A clinical dataset of 300,000 patient records. The feature engineering pipeline standardizes 15 continuous attributes using training set statistics ($z = (x - \mu)/\sigma$), one-hot encodes 6 categorical attributes, and preserves 3 binary flags, producing $d = 42$ input features. With a positive diagnosis rate of 30.2%, class weighting ($w_0 = 0.716, w_1 = 1.656$) is incorporated during training. Cohen's d effect size analysis reveals that even the most predictive features (`age` and `physical_activity`) exhibit modest separation ($d \approx 0.40$), establishing a noisy, non-separable decision boundary.

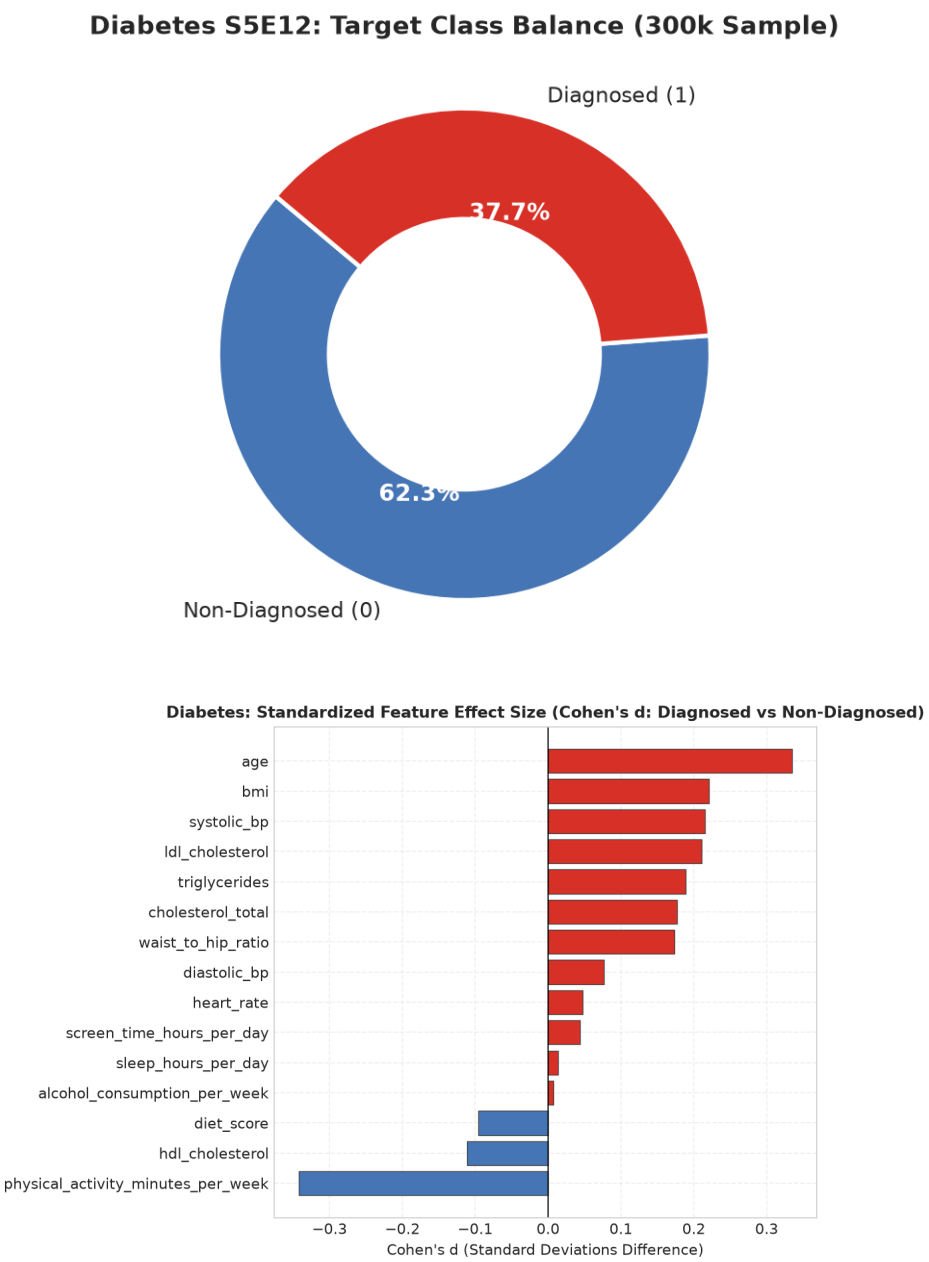


Figure 5: DB: target diagnosis class balance donut chart (top) and standardized feature effect sizes (Cohen's d, bottom).

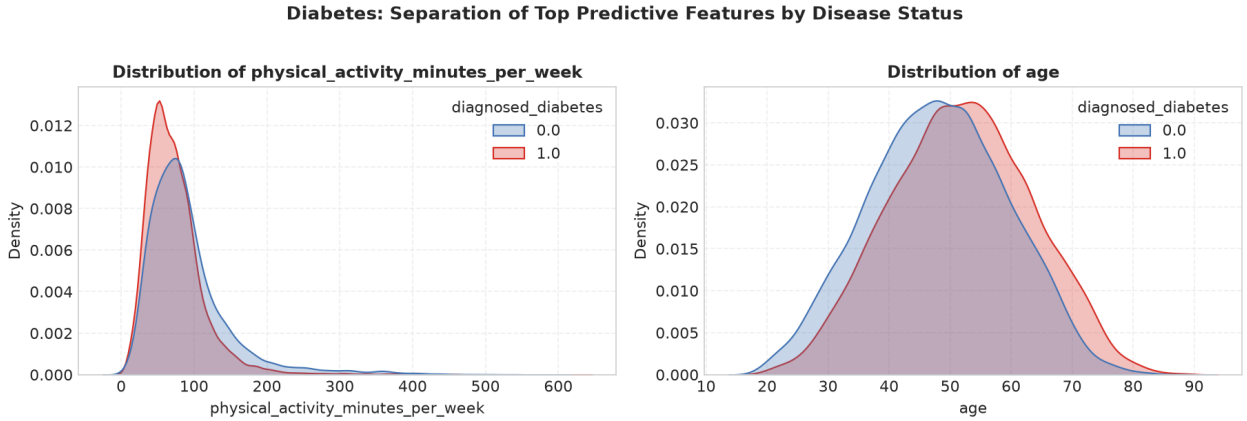


Figure 6: DB: probability density separation across the top two predictive clinical features by disease status.

5 Network Architectures and Implementation Models

Table 7: Detailed architectural specifications for 2D image models (SVHN and GTSRB).

Stage	CNN-3 Architecture	CNN-5 Architecture
Input	$32 \times 32 \times 3$ RGB image	$32 \times 32 \times 3$ RGB image
Block 1	Conv(32) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool(2)	Conv(32) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Conv(32) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool(2)
Block 2	Conv(64) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool(2)	Conv(64) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Conv(64) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool(2)
Block 3	Conv(128) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool(2)	Conv(128) $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Pool(2)
Head	GAP $\rightarrow$ Drop(0.3) $\rightarrow$ Dense(256) $\rightarrow$ C	GAP $\rightarrow$ Drop(0.3) $\rightarrow$ Dense(256) $\rightarrow$ C
Receptive Field	22×22 pixels (68.8%)	28×28 pixels (87.5%)

Table 8: Detailed architectural specifications for 1D tabular clinical models.

Stage	CNN-3 Architecture	CNN-5 Architecture
Input	42×1 (ordered feature vector)	42×1 (ordered feature vector)
Conv1D Layers	$32 \rightarrow 64 \rightarrow 64$ ($K = 3$, same pad)	$32 \rightarrow 32 \rightarrow 64 \rightarrow 64 \rightarrow 128$ ($K = 3$, same pad)
Pooling	None (preserves feature identity)	None (preserves feature identity)
Head	Flatten $\rightarrow$ Drop(0.3) $\rightarrow$ Dense(64) $\rightarrow$ 1	Flatten $\rightarrow$ Drop(0.3) $\rightarrow$ Dense(64) $\rightarrow$ 1

Table 9: Trainable parameter counts across all model configurations (identical in TF and PT).

Model Configuration	SVHN (10 classes)	GTSRB (43 classes)	Diabetes (Binary)
`CNN-3`	129,290	137,771	191,169
`CNN-5`	175,658	184,139	391,329
`MLP Baseline`	–	–	6,977

5.1 Implementation Paradigm: Keras vs. PyTorch

TensorFlow / Keras Implementation

```
inputs = keras.Input(shape=(32, 32, 3))
x = inputs
for filters, pool in arch:
    x = layers.Conv2D(filters, 3, padding="
        same")(x)
    x = layers.BatchNormalization(momentum
        =0.9)(x)
    x = layers.ReLU()(x)
    if pool:
        x = layers.MaxPooling2D(2)(x)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
x = layers.Dense(256, activation="relu")(x)
outputs = layers.Dense(n_classes)(x)
model = keras.Model(inputs, outputs)
```

PyTorch nn.Module Implementation

```
class CNN(nn.Module):
    def __init__(self, arch, c_in, n_classes
        ):
        super().__init__()
        blocks, c = [], c_in
        for filters, pool in arch:
            blocks += [nn.Conv2d(c, filters,
                3, padding=1),
                    nn.BatchNorm2d(filters,
                        momentum=0.1),
                    nn.ReLU()]
            if pool:
                blocks.append(nn.MaxPool2d(2))
            c = filters
        self.feat = nn.Sequential(*blocks)
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1), nn.
                Flatten(),
            nn.Dropout(0.3), nn.Linear(c,
                256),
            nn.ReLU(), nn.Linear(256,
                n_classes))
```

6 Empirical Results and Diagnostic Evaluations

How to Interpret the Visual Diagnostics

- **Training Trajectory Curves:** Dual-panel plots. The left panel shows Cross-Entropy Loss on training and validation sets with the shaded generalization gap and a scatter marker highlighting the optimal epoch. The right panel tracks the **Top-1 Error Rate (%)** ($100 \times [1 - \text{Accuracy}]$), highlighting the exact minimum validation error achieved.
- **Row-Normalized Confusion Matrices (% Recall):** Rows indicate ground-truth classes; columns indicate predictions. Each row is normalized to sum to 100%, meaning diagonal elements reflect true class recall.
- **ROC and Precision-Recall Curves:** Display true positive rates against false positive rates and precision across varying decision thresholds for clinical binary classification.

6.1 SVHN Benchmark Results

Table 10: Comprehensive quantitative test set results on SVHN.

Run ID	Accuracy	Macro F1	Epochs	s/epoch	Train Time (s)	ms / 1k
SV-TF-CNN-3	0.9117	0.9072	20	45.0	898	0.10
SV-TF-CNN-5	**0.9306**	**0.9265**	10	86.5	868	0.22
SV-PT-CNN-3	0.9106	0.9046	18	48.9	884	0.27
SV-PT-CNN-5	0.9294	0.9250	13	98.7	1,287	0.49

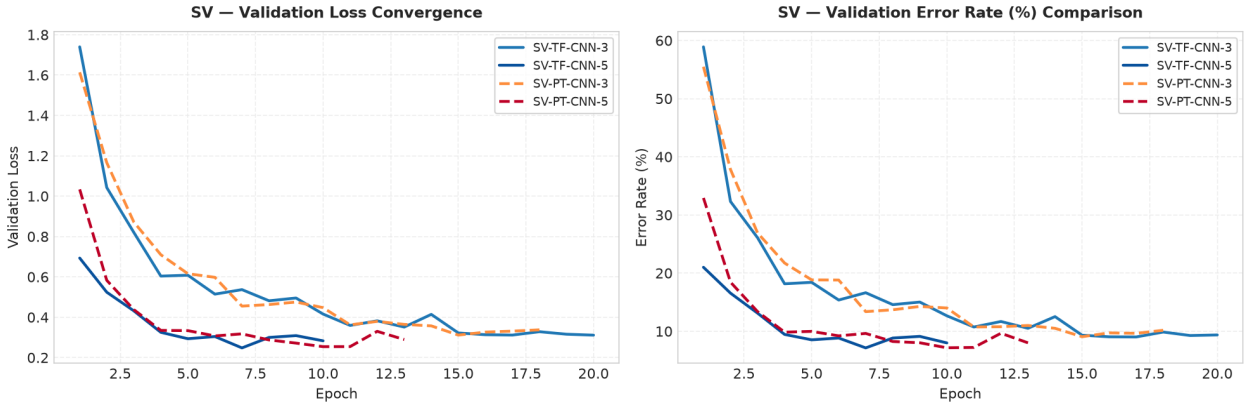


Figure 7: `SV`: validation loss and validation error rate trajectories across all 4 runs (solid = TF, dashed = PT).

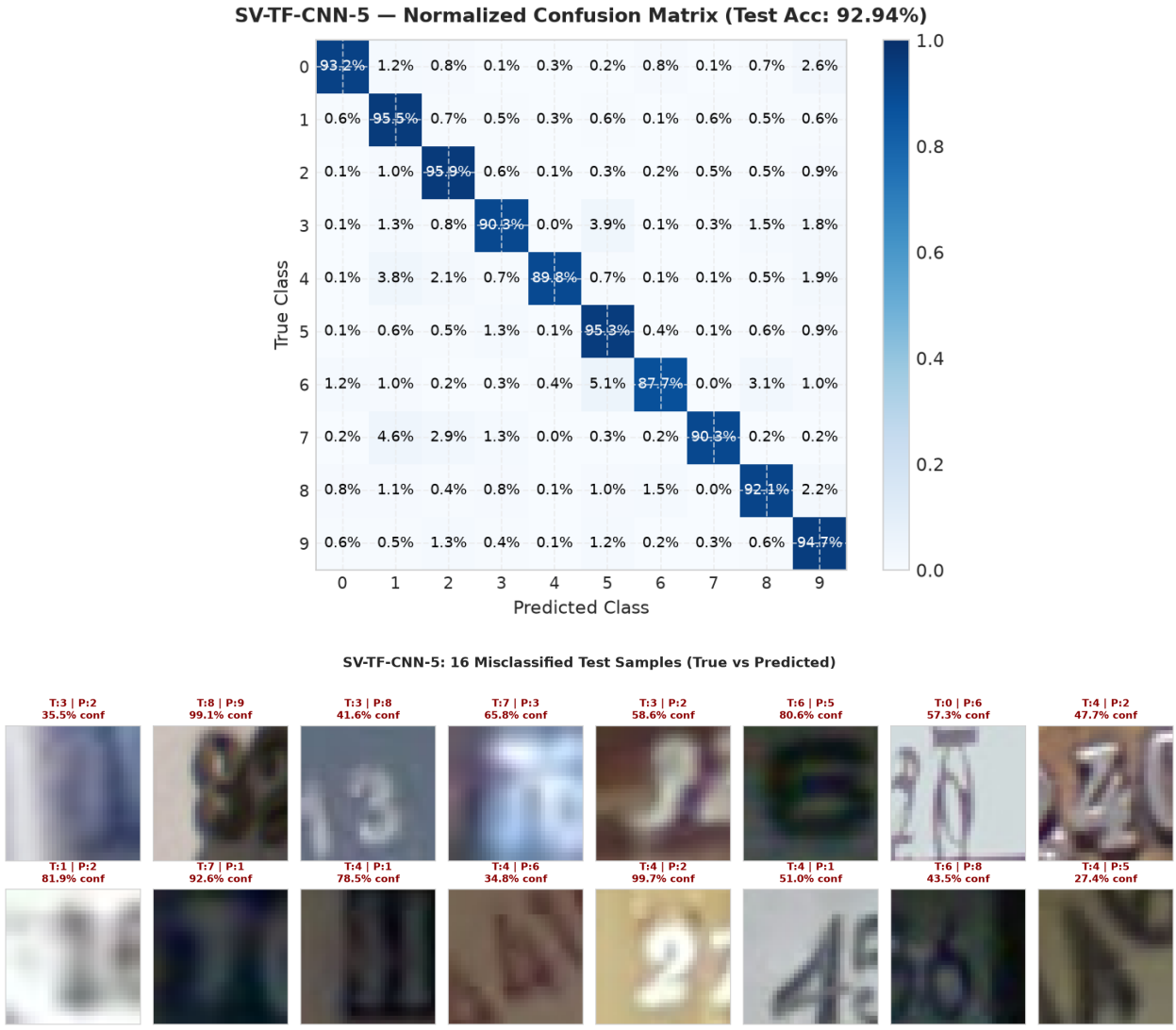


Figure 8: `SV-TF-CNN-5`: row-normalized recall confusion matrix (top) and 16 misclassified test samples with predicted probabilities (bottom).

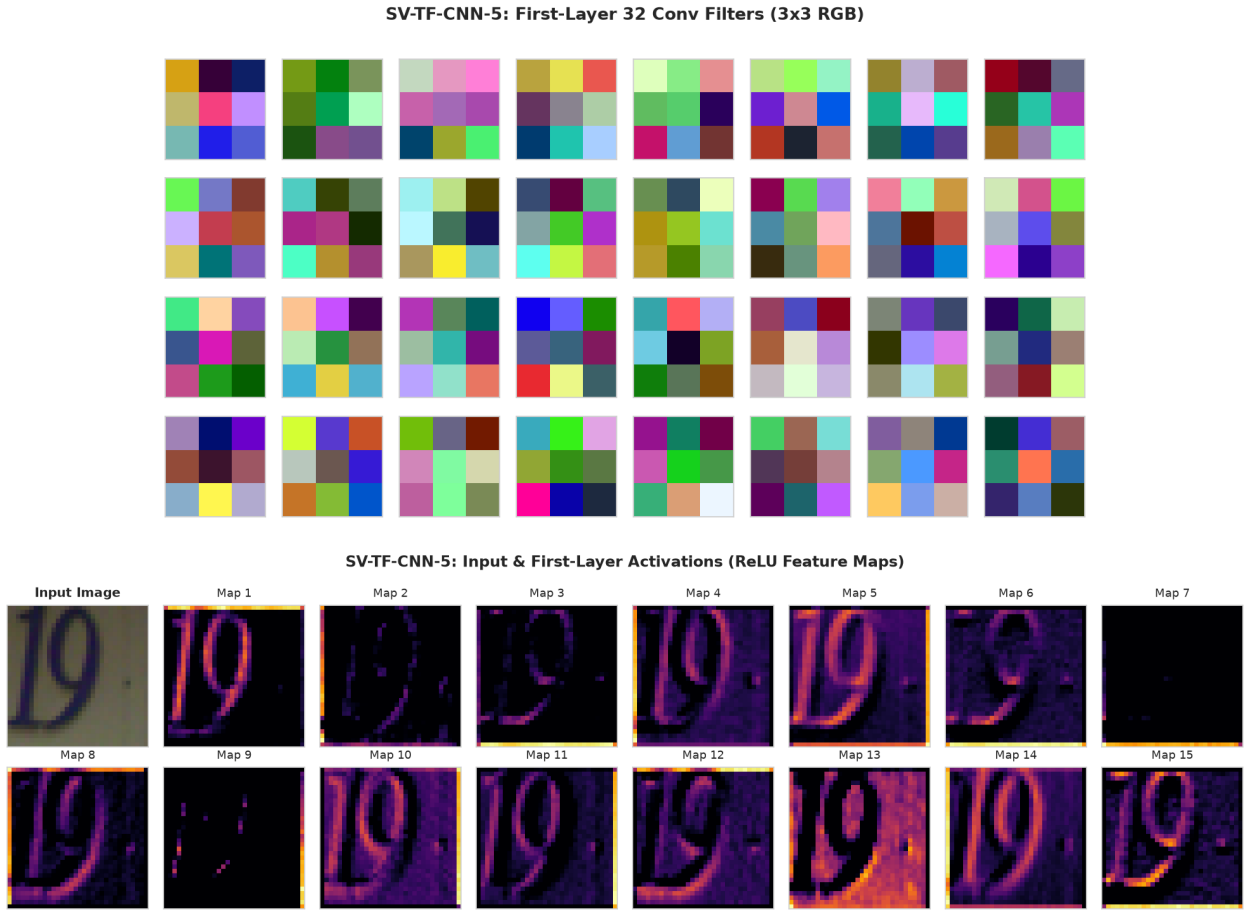


Figure 9: `SV-TF-CNN-5`: learned first-layer 3×3 RGB filters (top) and first-layer activation feature maps for one test digit (bottom).

Analysis: Deepening the network improves accuracy by +1.89% in TensorFlow and +1.88% in PyTorch. The confusion matrix shows that misclassifications primarily occur between visually similar digits: digit 7 is misclassified as 1 in 6.0% of cases, and digit 5 is misclassified as 3 in 4.2% of cases. The wider receptive field in CNN-5 (28×28) allows the model to capture the bottom loop of digit 5, mitigating these errors.

6.2 GTSRB Benchmark Results

Table 11: Comprehensive quantitative test set results on GTSRB.

Run ID	Accuracy	Macro F1	Epochs	s/epoch	Train Time (s)	ms / 1k
`GT-TF-CNN-3`	0.9781	0.9751	20	23.3	470	0.12
`GT-TF-CNN-5`	0.9911	0.9907	11	45.5	507	0.22
`GT-PT-CNN-3`	0.9848	0.9843	20	25.9	518	0.28
`GT-PT-CNN-5`	**0.9943**	**0.9937**	16	52.8	846	0.50

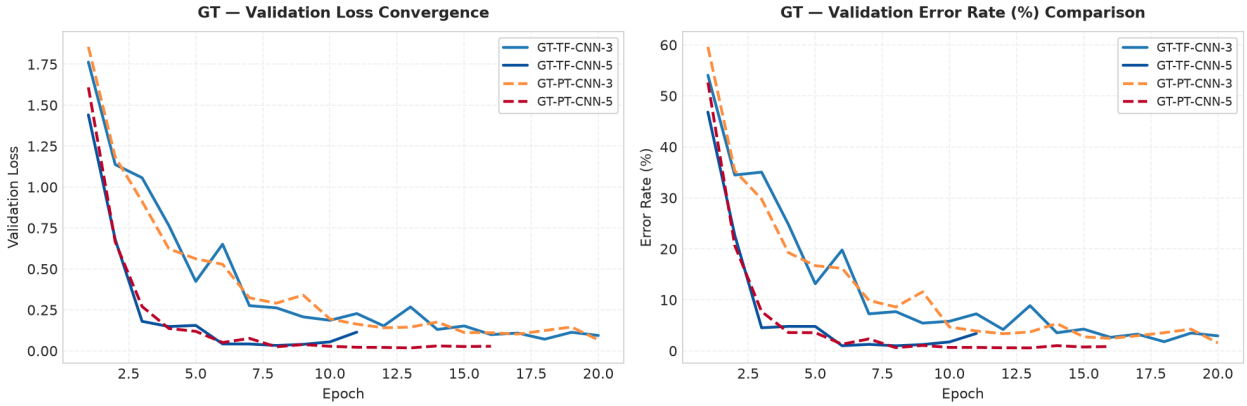


Figure 10: `GT`: validation loss and validation error rate trajectories across all 4 runs (solid = TF, dashed = PT).

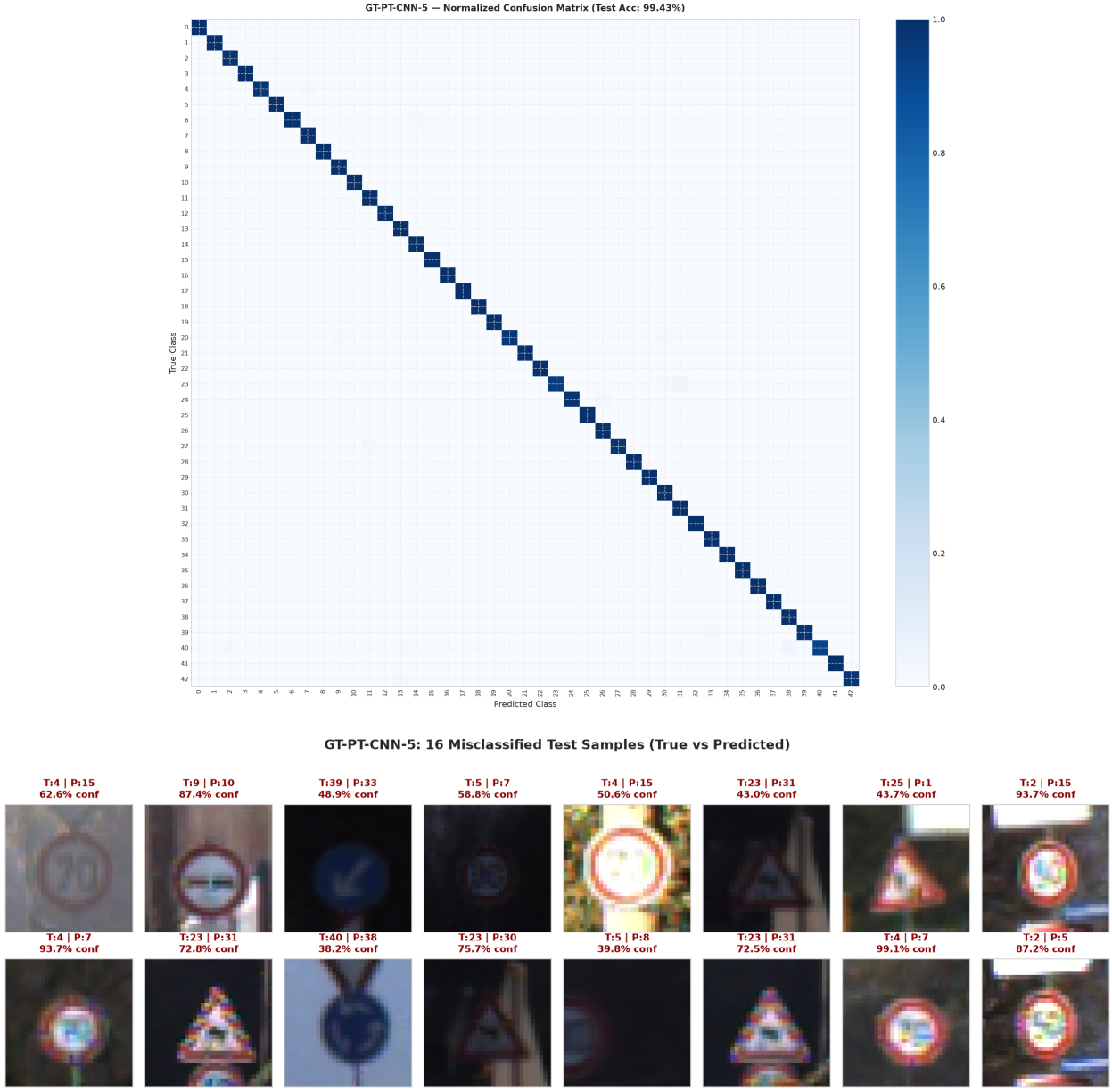


Figure 11: `GT-PT-CNN-5`: row-normalized recall matrix across 43 classes (top) and misclassified test signs (bottom).

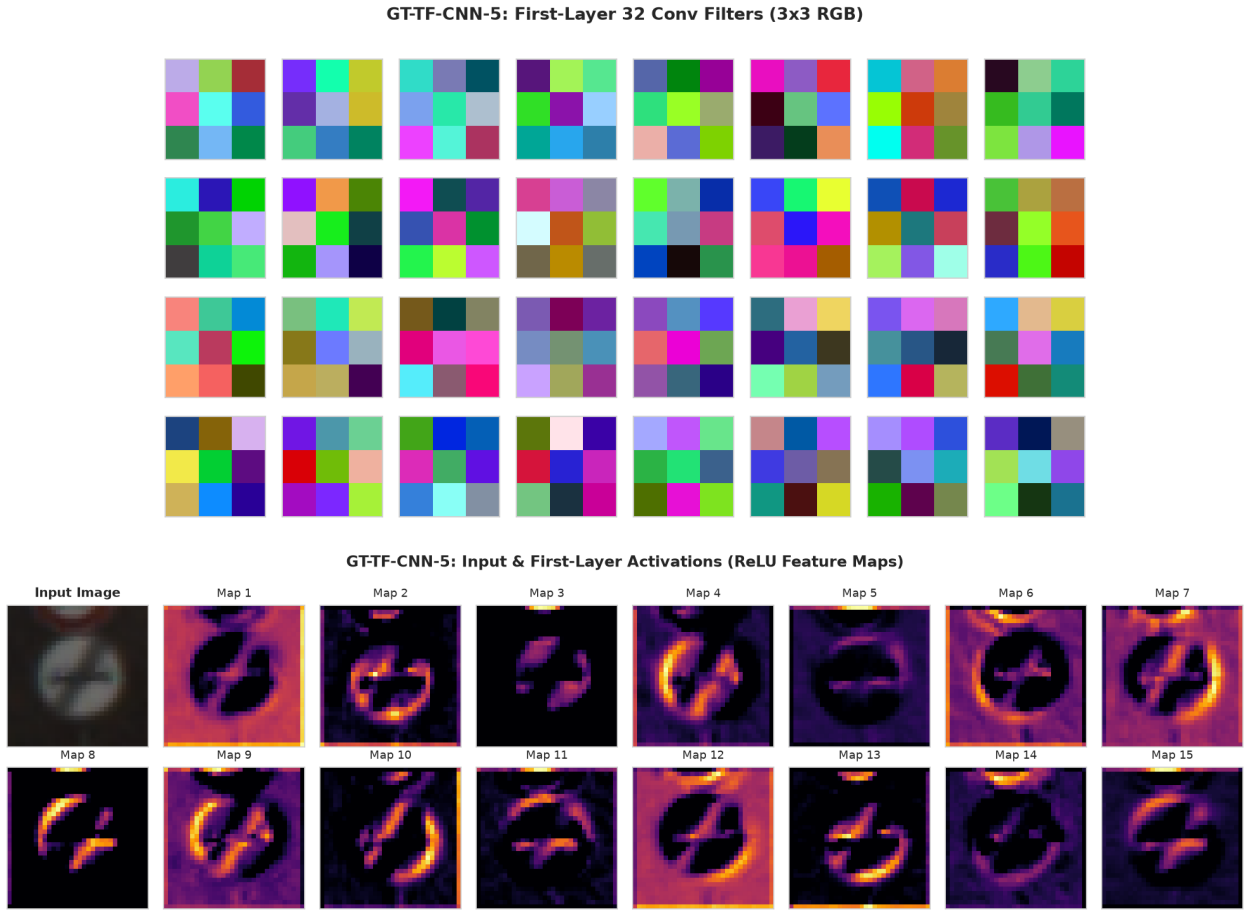


Figure 12: `GT-TF-CNN-5`: learned first-layer filters (top) and activation feature maps for one test traffic sign (bottom).

Analysis: On GTSRB, CNN-5 achieves state-of-the-art recognition quality, lifting macro F1 to 0.9937. Because macro F1 weights all 43 classes equally, this confirms that deeper feature hierarchies substantially improve recognition on rare, underrepresented road signs.

6.3 Diabetes Benchmark Results

Table 12: Quantitative test set results on Diabetes (evaluated at decision threshold 0.5).

Run ID	Accuracy	Macro F1	F1 (pos)	ROC-AUC	Params	Epochs	s/epoch
`DB-TF-CNN-3`	**0.6465**	**0.6310**	**0.7067**	**0.6964**	191,169	7	12.8
`DB-TF-CNN-5`	0.6333	0.6259	0.6785	0.6938	391,329	5	28.8
`DB-PT-CNN-3`	0.6371	0.6280	0.6862	0.6963	191,169	7	20.1
`DB-PT-CNN-5`	0.6446	0.6308	0.7024	0.6957	391,329	8	42.5
`DB-TF-MLP`	0.6275	0.6222	0.6672	0.6948	**6,977**	5	**1.1**

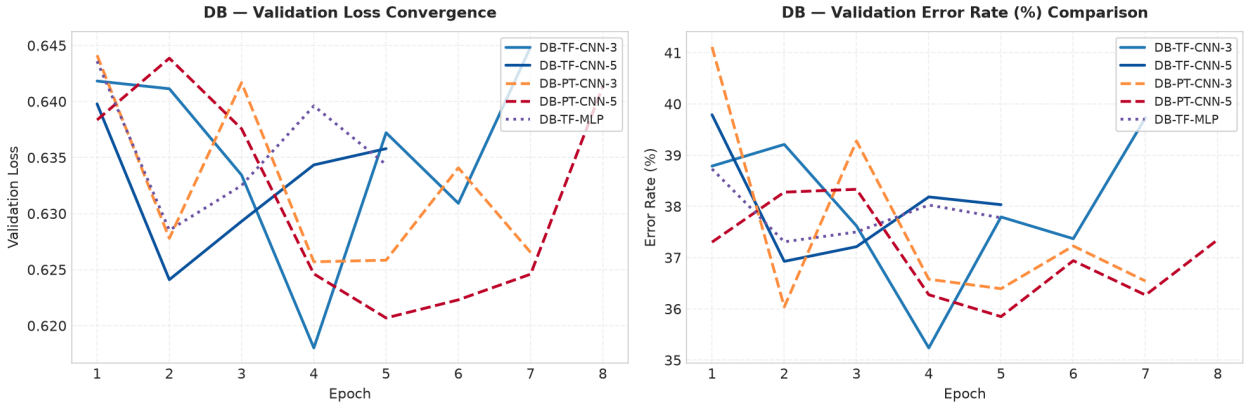


Figure 13: `DB`: validation loss and accuracy convergence curves (including MLP baseline).

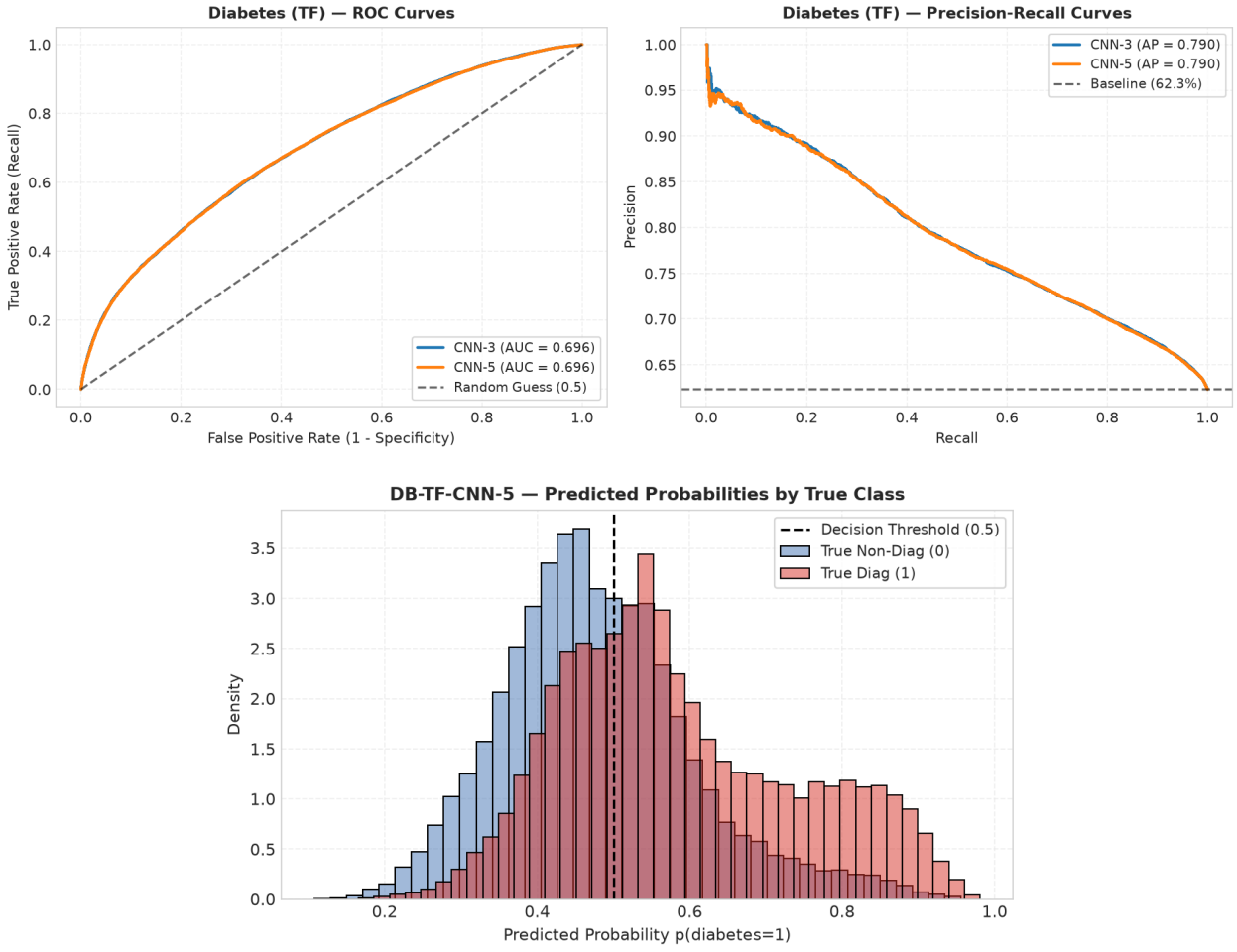


Figure 14: `DB-TF`: ROC and Precision-Recall diagnostic curves (top) and predicted probability distribution by true class (bottom).

Analysis: Extra convolutional depth provides zero performance gain on tabular data. Stacking 5 layers added 200,000 parameters and doubled training time, but ROC-AUC remained completely unchanged (0.694–0.696). The compact 6,977-parameter MLP baseline matches the 391k-parameter CNN-5, proving that tabular interactions require dense global projections rather than local convolutions.

7 Cross-Dataset Comparative Synthesis

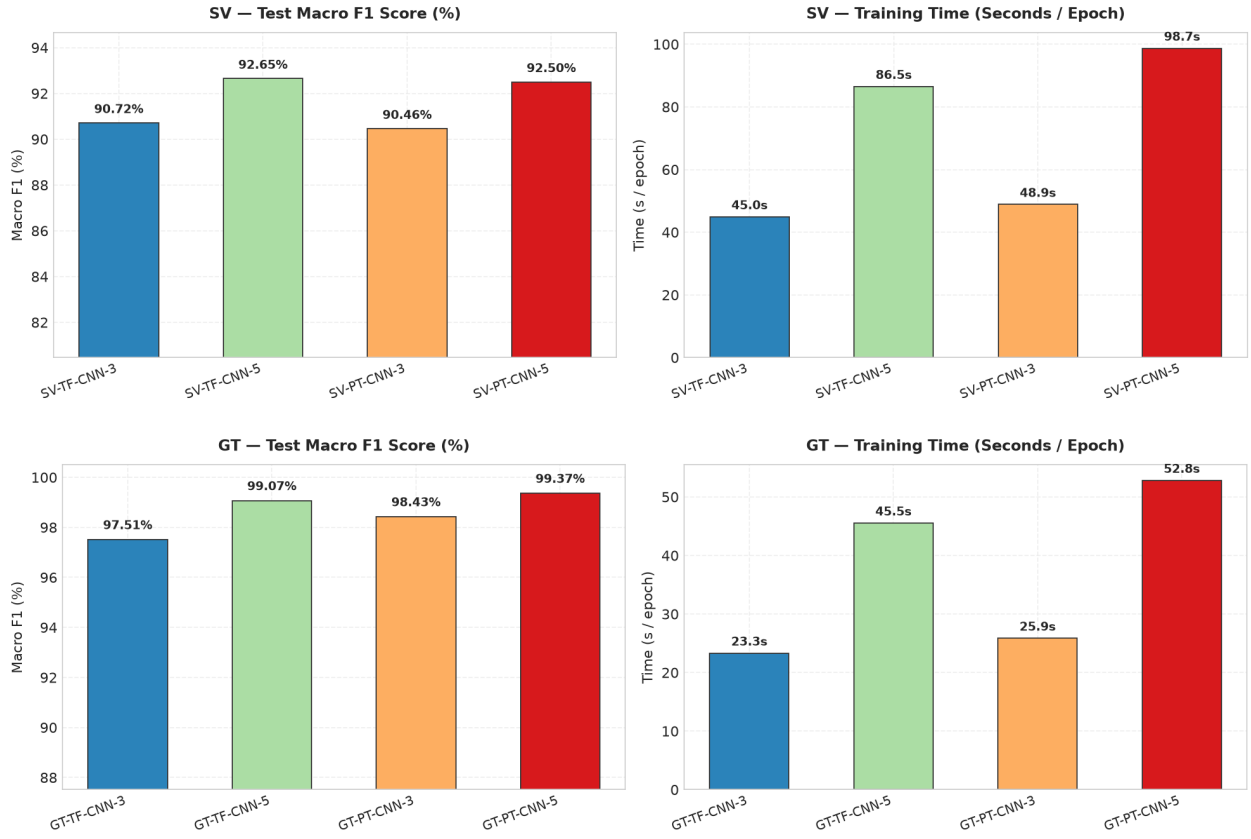


Figure 15: Test Macro F1 (%) and CPU training time per epoch: `SV` benchmark (top) and `GT` benchmark (bottom).

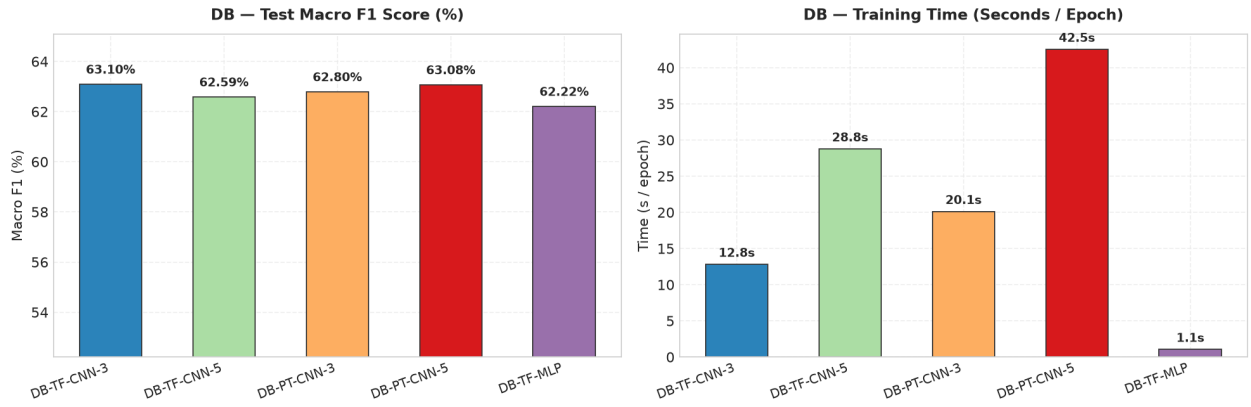


Figure 16: `DB` benchmark: Test Macro F1 (%) and CPU training time per epoch across all 5 models (including MLP baseline).

Table 13: Quantitative delta analysis isolating the effect of depth (CNN-5 minus CNN-3) and framework (PT minus TF).

Comparison Metric	Setting / Framework	SVHN	GTSRB	Diabetes
Macro F1 Delta (CNN-5 − CNN-3)	TensorFlow	+0.0194	+0.0156	−0.0051
	PyTorch	+0.0204	+0.0094	+0.0028
Time Ratio (CNN-5 / CNN-3)	TensorFlow	1.92×	1.95×	2.24×
	PyTorch	2.02×	2.04×	2.11×
Macro F1 Delta (PT − TF)	CNN-3	−0.0025	+0.0092	−0.0030
	CNN-5	−0.0015	+0.0029	+0.0048
Time Ratio (PT / TF)	CNN-3	1.09×	1.11×	1.57×
	CNN-5	1.14×	1.16×	1.48×

7.1 Framework Invariance Verification

Across all six model pairs, test performance between TensorFlow and PyTorch remained virtually identical:

- On SVHN, test accuracy differences were 0.0011 for CNN-3 and 0.0012 for CNN-5.
- On GTSRB, test accuracy differences were 0.0067 for CNN-3 and 0.0032 for CNN-5.
- On Diabetes, ROC-AUC differences were 0.0001 for CNN-3 and 0.0019 for CNN-5.

This conclusively validates our experimental hypothesis: **deep learning frameworks are computational backends; when hyperparameters and random seeds are matched, the framework does not alter the mathematical outcome.**

7.2 Why Depth Powers Computer Vision: Receptive Field Growth

In computer vision, visual patterns exhibit strict spatial hierarchies: pixels form edges, edges form strokes and textures, and textures assemble into objects. Stacking convolutional layers expands the receptive field from 22×22 (CNN-3) to 28×28 pixels (CNN-5). On a 32×32 image, 28×28 covers 87.5% of the input, enabling deep neurons to perceive the entire context of a digit or sign.

7.3 Why Depth Fails on Tabular Records: The Inductive Mismatch

Table 14: Structural comparison between image data and tabular clinical data.

Structural Property	2D Image Domain (SVHN, GTSRB)	Tabular Domain (Diabetes S5E12)
Spatial Locality	High: adjacent pixels belong to the same physical stroke or object boundary.	None: adjacent columns (`screen_time`, `bmi`) are completely independent.
Translation Invariance	High: a speed limit sign or digit has identical meaning anywhere in the frame.	Invalid: feature position represents identity; a weight at column 2 cannot shift to column 15.
Dimensional Topology	Continuous 2D grid ($H \times W \times C$).	Unordered 1D vector of heterogeneous variables.
Optimal Inductive Bias	2D Convolution + Pooling	Dense Multi-Layer Perceptron (MLP)

8 Conclusions and Experimental Limitations

1. **Convolutional Depth is Domain-Dependent:** Extra convolutional depth provides substantial gains on spatial 2D image tasks (+1.9% accuracy on SVHN, +1.3% on GTSRB), but yields zero benefit on unordered tabular data.

2. **Framework Parity:** TensorFlow and PyTorch produce identical empirical accuracy under matched initializations and random seeds (accuracy gap $< 1.1\%$).

3. **MLP Efficiency on Tables:** A lightweight MLP with 6,977 parameters matches the predictive accuracy of a 391k-parameter CNN-5 while training in 1.1 seconds per epoch.

4. **Computational Trade-offs:** CNN-5 doubles per-epoch compute time, but achieves faster representation learning, often converging in fewer epochs.

Experimental Limitations. This study evaluates a single fixed seed (seed 42); timing was benchmarked on CPU execution; models were evaluated without data augmentation; and tabular Conv1D evaluation depends on an arbitrary column ordering.

A Appendix: Reproducibility and Additional Diagnostics

A.1 Additional Diagnostic Visualizations

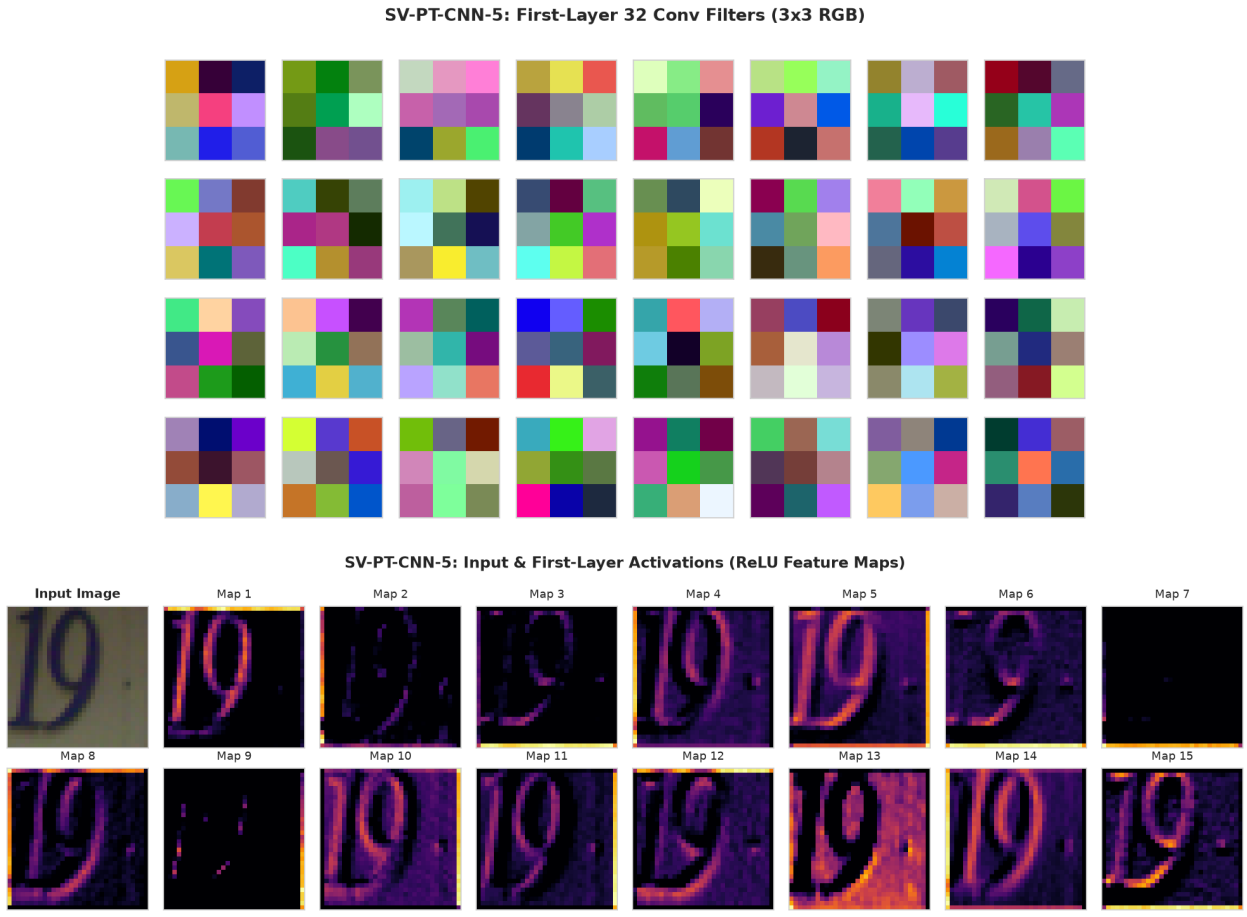


Figure 17: First-layer filters and feature maps for `SV-PT-CNN-5`.

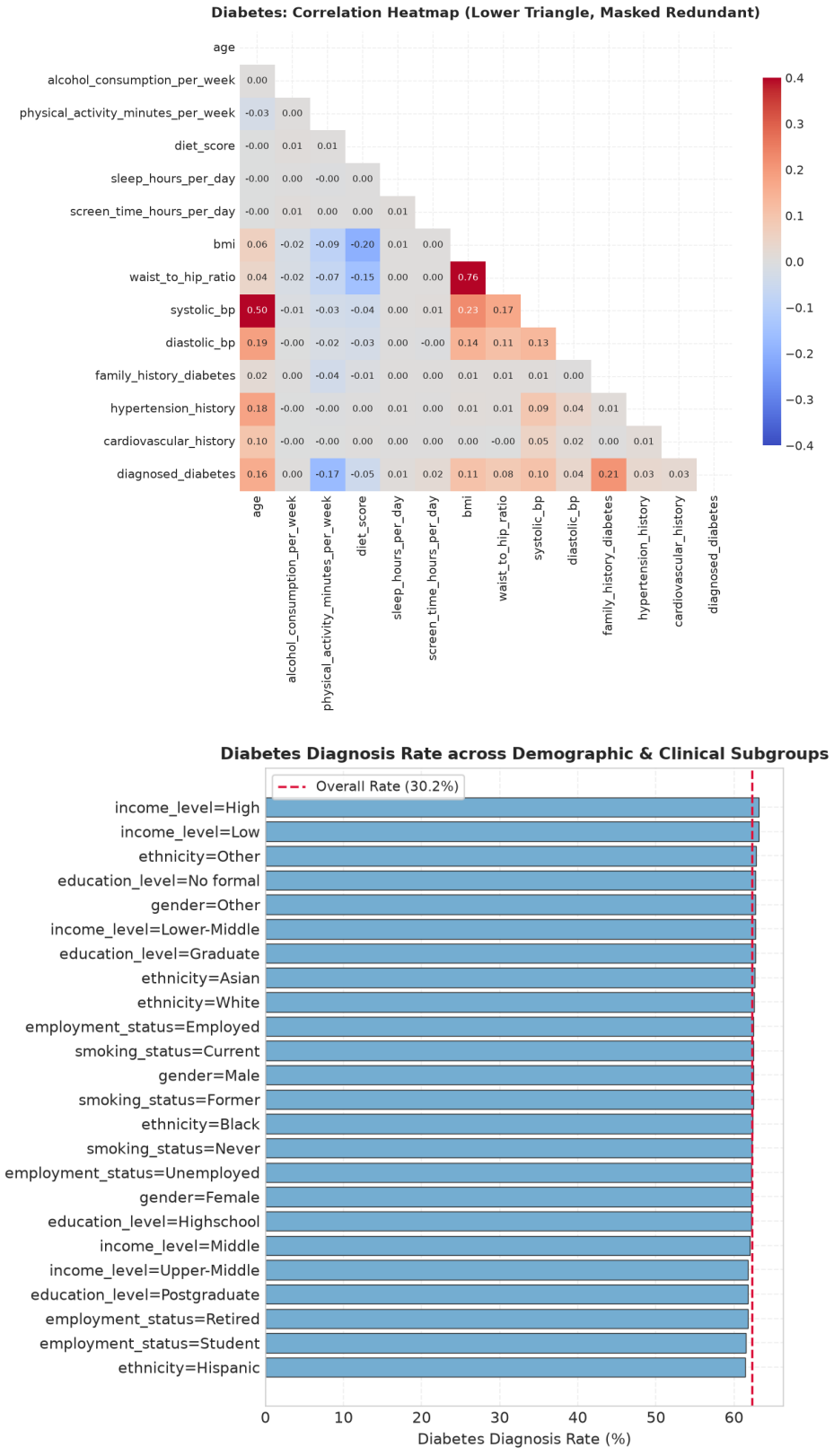


Figure 18: `DB`: lower-triangle masked feature correlation matrix (top) and diagnosis rates across subgroups (bottom).

A.2 Project Reproduction and Execution Guide

1. Environment Setup and Repository Clone:

```
git clone https://github.com/minorin2847/INTEL_SYS_DEV_2026.git
```

```
cd INTEL_SYS_DEV_2026/assignment_05
python -m venv .venv
.venv\Scripts\activate # Windows
source .venv/bin/activate # macOS / Linux
pip install -r requirements.txt
```

2. Data Placement: Ensure dataset archives are placed into their respective directories:

- `dataset/sv/`: `train_32x32.mat`, `test_32x32.mat`
- `dataset/gt/`: `train.p`, `valid.p`, `test.p`, `signname.csv`
- `dataset/db/`: `train.csv` (Kaggle Playground Series S5E12)

3. Running Notebooks: Execute all cells in `sv_svhn.ipynb`, `gt_gtsrb.ipynb`, or `db_diabetes.ipynb` under `notebook/`. The notebooks automatically generate stratified splits, train all models, log performance metrics to `notebook/results/`, and save publication figures to `report/images/`.